Hands-On 2: Segment Trees and Range Queries

Competitive Programming and Contests (2025/2026) – Lorenzo Ferrari

1 Overview

This assignment explores tree-based data structures for efficient range queries and updates. This report outlines the core algorithmic logic and complexity for Problem 1 (`is_there`) and Problem 2 (`min_and_max`). This report has been generated by Antigravity with Gemini 3.8 Flash and modified by the author.

2 `is_there`: Range Frequency Queries via Merge Sort Tree

Given N segments $[l_i, r_i]$, we determine the coverage of each position and process M queries (l, r, k), each asking whether any index $x \in [l, r]$ is covered by exactly k segments.

- **Algorithmic Logic**:
  - **Coverage via Difference Array**: We compute the number of covering segments at each position in $\mathcal{O}(N)$ time using a difference array followed by a single prefix sum sweep.
  - **Merge Sort Tree**: To support point-value existence queries over arbitrary subranges, we build a **Merge Sort Tree** over the coverage array. Each node represents a range and stores its values in sorted order, constructed bottom-up by merging the sorted lists of its children.
  - **Range Queries**: An interval query $[l, r]$ decomposes into $\mathcal{O}(\log N)$ canonical disjoint nodes. On each canonical node, we perform a binary search for value k, short-circuiting immediately if k is found.
- **Complexity**:
  - **Preprocessing & Construction**: $\mathcal{O}(N)$ for coverage prefix sums and $\mathcal{O}(N \log N)$ time to construct the tree, merging sorted arrays across $\mathcal{O}(\log N)$ levels.
  - **Query Time**: Each query inspects at most $\mathcal{O}(\log N)$ canonical nodes; binary search takes $\mathcal{O}(\log N)$ time per node, yielding $\mathcal{O}(\log^2 N)$ worst-case time per query.
  - **Total Complexity**: $\mathcal{O}(N \log N + M \log^2 N)$ time and $\mathcal{O}(N \log N)$ auxiliary space.

3 `min_and_max`: Range Chmin Updates and Range Max Queries

Given an array of N integers, we maintain it under M operations: range updates capping elements to at most t ($a[i] \leftarrow \min(a[i], t)$ for $i \in [l, r]$), and range maximum queries ($\max_{i \in [l,r]} a[i]$).

- **Algorithmic Logic**:
  - **Distributive Compatibility**: The chmin operation is compatible with range maximum aggregation because $\min$ distributes over $\max$:

    $$\max(\min(x_1, t), \min(x_2, t)) = \min(\max(x_1, x_2), t)$$

    Applying an upper bound cap t to a subtree with maximum M directly updates its maximum to $\min(M, t)$, without requiring knowledge of individual leaf values.
  - **Lazy Tag Composition & Propagation**: A **Segment Tree with Lazy Propagation** maintains the range maximum and a pending upper-bound tag per node. Successive lazy updates compose monotonically via minimum ($t_{\text{new_tag}} = \min(t_{\text{old}}, t_{\text{new}})$). Tags are held at canonical nodes and propagated down to children only when recursing into subtrees during updates or queries.
- **Complexity**:
  - **Construction**: $\mathcal{O}(N)$ linear time and space.
  - **Operations**: Both range updates and range maximum queries visit $\mathcal{O}(\log N)$ canonical nodes, performing $\mathcal{O}(1)$ work at each node via lazy tag application and propagation.
  - **Total Complexity**: $\mathcal{O}(N + M \log N)$ time and $\mathcal{O}(N)$ space.